

THE OPEN-SOURCE AUDIT

A Structured Examination of GIT

Chosen Software:

GIT — GPL v2 License

Course: Open Source Software | VITyarthi

Submitted By:

Name: *Nihal Vats*

Registration Number: *24BCE10837*

Program: BTech

Branch: *Computer Science*

Semester: *V*

Submitted To:

Faculty Name: Dr Ajay Kumar

Course Code: *CSE0002*

Institution:

VIT Bhopal University

March 2026

License	First Released
General Public License	2005 by Linus Torvalds

Table of Contents

1. Introduction	3
2. Part A1 — Origin Story and Problem Context	4
3. Part A2 — License Analysis	6
4. Part A3 — Ethics of Open Source	8
5. Part B — Linux Footprint	10
6. Part C — The FOSS Ecosystem	15
7. Part D — Open Source vs Proprietary	19
8. Shell Scripts	21
9. References	26

1. Introduction

Most of the software we use every day, including websites, apps, and operating systems, wasn't created by one big company that keeps it locked away. It was built by regular people who shared their work openly so others could use it, learn from it, and improve it. This idea is called open-source software.

For this project, I chose Git, a tool used by nearly every software developer in the world to track changes in their code. Linus Torvalds created Git in 2005 and released it under a license called GPL v2. This means anyone can use it, read its code, modify it, and share it freely.

In this report, I will look at Git from five angles: its origins and purpose, what its license allows, how it works on a Linux system, its connections to other tools and communities, and how it compares to paid alternatives.

As someone studying software, understanding tools like Git is important. Everyone uses it, and the story behind its creation provides valuable insights into how good software is made and shared.

Git and similar tools are crucial. We need to know why they were created and why people give them away for free. When learning about software, you can't simply skim this. In actuality, it serves as the foundation for your education. Open-source tools are used in nearly every line of code written today. Every developer should also understand the origins of these tools and their true objective. Because tools like Git are an integral part of their work, developers should be conscious of the values they uphold.

2. Part A — Origin and Philosophy

A1. The Problem GIT Was Created to Solve

Git was not planned. It was not developed by a team with funding and a product plan. It was developed by one person in about ten days because he had no other choice. To understand why there is a Git version control system, it is necessary to understand what was happening in April 2005.

The Way Linux Kernel Was Handled Before Git

Linux kernel is an enormous collaborative project. There are thousands of programmers around the world contributing code to this project. In the early 2000s, handling this was turning out to be a serious issue.

Programmers were sending code changes to Linus Torvalds via email in a form known as 'patches.' Linus and his team were manually reviewing and applying these changes. There was no system in place to track changes and reasons behind changes.

And so, in 2002, Linus made a decision: he began to use a version control system called BitKeeper to manage the kernel's code.

The problem with BitKeeper

BitKeeper was a good version control system. But there was one major problem: it was a proprietary system. This means that the company that owned BitKeeper, BitMover, owned it completely. You could not read its code, change its code, or use it as you wanted to. They gave it away for free to open-source projects, but as soon as they wanted to stop giving it away for free, everyone else who was using it would be stuck.

There were a lot of people within the Linux community who had a problem with this from the very beginning. It just seemed wrong to use a closed-source system to develop what is arguably the most famous open-source system in the world.

What actually broke things

A developer by the name of Andrew Tridgell, in the month of April 2005, began reverse engineering the BitKeeper protocol. This is essentially the process of reverse engineering the BitKeeper protocol, i.e., understanding it from the outside without access to the source code.

This, in turn, led BitMover to do what everyone was worried they might do: revoke free access to BitKeeper for the Linux project.

Just like that, the entire Linux kernel development team was left without a version control system.

Linus builds Git

He also had a look at the existing open source solutions. CVS and SVN were the primary ones. He wasn't satisfied with either of them. He felt they were slow, limited, and centralized, which wasn't appropriate for the kernel.

He then decided to roll his own solution.

His requirements were simple but stringent:

- It had to be fast. The kernel is enormous, and thousands of changes are happening all the time.
- It had to be distributed. Every developer had to have the complete history on their own machine, and they didn't need a central server.
- It had to ensure data integrity. This had to be achieved using something called SHA-1 hashing. This way, if data changes or gets corrupted, it would be caught.
- It had to be good at branching and merging. The kernel had hundreds of parallel lines of development at the same time.

In about ten days, he had a working version of Git. The first ever commit to the Git repository happened on April 7, 2005. Within months, the Linux kernel completely migrated to Git.

He named it "Git" which is British slang for a stupid or annoying person. His reason for choosing the name is, "I name all my projects after myself—first Linux, now Git."

Why he made it open source

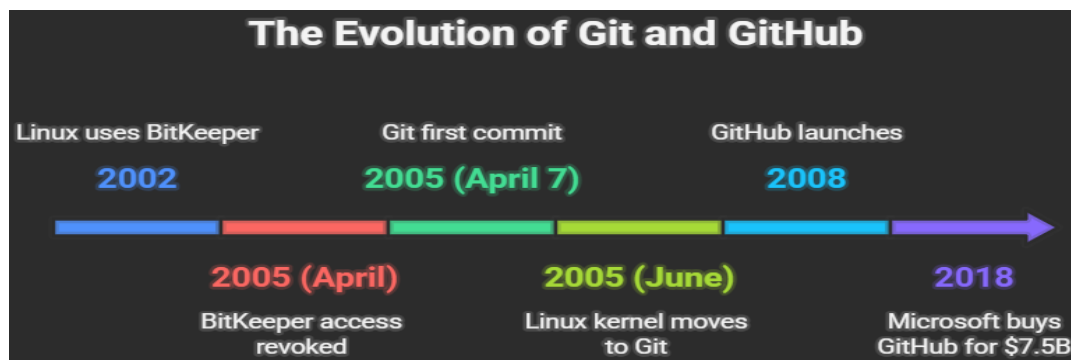
He chose to make it open source and license it under GPL v2, just like Linux. This is because "the whole problem started because the community was using a proprietary tool. If we'd been using a proprietary tool, the same thing would have happened to us."

By making it open source, Linus ensured that no company could ever take it away from the developers. It is open to anyone to read, use, or modify it.

What Git has become

What began as a personal solution to a very specific problem has grown to become something much, much bigger. Today, it is used by essentially every software developer on the planet. GitHub, a service based on Git, is owned by Microsoft, acquired in 2018 for 7.5 billion dollars. GitLab, Bitbucket, and many other services rely on Git. Every major tech company in the world uses it inside their company.

None of this would exist if one developer hadn't gotten frustrated and wanted to make something better, and share it.



3. Part A2 — The License: What GPL v2 Actually Says

Git is distributed under the GNU General Public License version 2, or GPL version 2. This is not just a legal document; it is a statement of what the software is for and what it is giving you.

The Four Freedoms of Free Software

A developer named Richard Stallman created the idea of the “four freedoms” – the basic rights that every user of free software should have. GPL v2 gives all four of them.

Freedom 0	You can use Git for anything you want. Personal projects, commercial work, government systems – no restrictions.
Freedom 1	You can look at Git's source code and understand how it works. You can change it however you like.
Freedom 2	You can share copies of Git with anyone.
Freedom 3	You can share your modified version of Git with others – as long as you give them the same freedoms.

The last point is the most important. GPL v2 is a copyleft license. It uses copyright law to preserve the openness of the software. If you take the Git code and you want to distribute a modified version, you have to distribute it under GPL v2 as well. You cannot close it.

Can a company take Git, modify it, and sell it secretly?

No, and this is the point of GPL v2.

If any company takes the Git source code, changes it, and distributes the modified version of the code for free or for any other price, they have to distribute their source code under GPL v2. They cannot close their code.

They can definitely create applications using Git and sell those applications for money. That's exactly what GitHub does. But the Git source code should always be open. It should never be closed.

GPL vs MIT – What's the difference?

These are the two most common open-source licenses, and they represent two different ways of thinking.

The **MIT License** is very permissive. It basically states: you may use this code for anything you want. You don't have to pass any of your results back.

The GPL adds a constraint: you must pass back anything you create using this code. You must pass back the code, and you must pass back the code under the GPL.

One way to think of the two is this: the MIT License is saying, "Here, take this. Do with it what you want." The GPL is saying, "*Here, take this. Do with it what you want. But pass it back.*"

If I were to have to choose a license for something I have created, I would choose the GPL
General Public License | Open Source Software CoursePage

for something I want to stay open. I would choose the MIT License for something I want to be used as much as possible, even if it means companies using it in their own products.

For the Git program, the GPL license was definitely the way to go. That is because the reason for creating Git was because a proprietary program was revoked.

Feature	GPL v3	MIT
Share source of changes?	Required (copyleft)	Not required
Commercial use allowed?	Yes, but changes must stay open	Yes, freely
Patent protection?	Explicit grant	Implied only
Complexity	High	Very low

Free as in Beer Vs Free as in Freedom

This is a phrase you'll hear a lot in open-source discussion and it's worth understanding properly.

"Free as in free beer" means it doesn't cost anything. You don't have to pay for it. But the person who is providing it to you still owns it. They can still take it away from you, change the rules, or terminate it whenever they want.

"Free as in freedom" means you own it. You have rights to it. You have the right to read it, change it, and distribute it. The price tag doesn't matter. What matters is that you own it, not the programmer.

BitKeeper was free as in beer. That means it didn't cost anything to use it. But it wasn't free as in freedom. That means they completely owned it, and when they terminated it, we had nothing.

Git is free as in beer and free as in freedom. That means it doesn't cost anything to get it, and at the same time, you own it completely. But I think the more important of the two is the second kind of free. The freedom is what's going to save the community.

4. Part A3 — The Ethics of Open Source

Open source isn't just about licenses and code. It's about values – about how knowledge should be shared, who should have access to tools, and what's fair.

Should All Software Be Open Source?

The Case For

There is a strong case for this. With open code, anybody can inspect it for security flaws. Knowledge is spread quickly. Communities can enhance a piece of software quickly, much quicker than a single company could. Even if a company drops a piece of software, a community can keep it alive.

Git is a good example. The moment it was removed, a community would come to a halt because it is closed-source. If something like this happened to Git, anybody could just use it and continue it themselves.

But there is a case for why software should not be forced open. Some software takes years and a huge investment to make. Companies need to make money to survive. Some software may even be less secure if its code is open for anybody to read for malicious intent. And a creator has a right to their work.

The honest answer is: software that everyone relies on, like operating systems, version control, compilers, web servers, etc. – they probably ought to be open source. Software that is more specific or developed commercially can obviously remain private. The key is that software that everyone relies on ought not to be owned or controlled by a single company.

Is It fair for companies to profit from free community work?

It's a very uncomfortable question, to be sure. Companies like Google, Facebook, and Microsoft rely on Linux, which was created by thousands of free volunteers, to power systems worth hundreds of billions of dollars. Red Hat has built a company worth \$34 billion largely on free software.

Is it ethical?

It's complicated. Red Hat does give back to Linux. Google sponsors the Summer of Code, supporting open-source programmers every year. These companies aren't just takers; they're also givers.

However, the disparity is undeniable. A programmer who contributes crucial code to Git or Linux makes nothing, while Google makes billions from his or her work. This is an issue the open-source world is still working through, and it is a real ethical issue, not a resolved one.

My take on this is that it is perfectly fine to use open-source software for commercial purposes, as this is essentially the purpose of open-source software – to make it freely available for use. However, it is not fine to use open-source software and avoid giving anything back to the open-source community, especially if you're making billions of dollars from it.

Standing on the Shoulders of Giants

This means that everything that is new is built on top of what has come before. This is true in the literal sense in the field of computer programming.

Git itself is a result of decades of ideas that have come before it: Unix, C, SHA-1, CVS, SVN, and the GPL license. Linus didn't invent version control. He just invented a better version of it by learning from all of what had come before.

Open source allows this kind of building to occur in a much easier fashion. This is why open-source projects tend to be better than their proprietary counterparts: they have had thousands of minds contribute to them, rather than just hundreds.

The burden of this is to contribute back. If you have benefited from something, then it's what you must do to keep the cycle going.

Part B — Linux Footprint

This section describes how Git resides on a Linux system. This section will include how it is installed, where it resides in terms of location on the system, who it is owned by in terms of user and group, whether it is run in the background, and how security patches are applied to the system. All of the information in this section was done using Ubuntu 22.04 in VirtualBox.

Installation Method

It is available in Ubuntu's official repository. This means users don't need to download anything, and there is no need to compile anything from source. It can be installed in one command using apt, which is Ubuntu's own package manager. This is the recommended way for Ubuntu users, as it includes all dependencies and will update Git when you update the system.

Git can be installed in multiple ways depending on the Linux distribution being used.

The table below shows the available options:

Method	Available for Git?	Command	Notes
apt (Ubuntu/Debian)	Yes	sudo apt install git	Easiest method, recommended for Ubuntu
RPM (Fedora/RHEL)	Yes	sudo dnf install git	For Red Hat based systems
Compile from Source	Yes	make install	For advanced users who need latest versions
Snap	Yes	sudo snap install git-ubuntu	Alternative method for Ubuntu

For this project, the apt method was used on Ubuntu 22.04. The commands used to install and verify Git were: `sudo apt update`, then ***sudo apt install git -y***, and finally ***git --version*** to confirm the installation was successful.

Key Directories on a Linux System

After installation, the files installed by Git will be found in a variety of standard Linux directories. Linux adheres to a standard known as the Filesystem Hierarchy Standard, which dictates where different types of files should be stored on a Linux system. The command ``dpkg -L git`` lists every single file installed by the Git package and where they're stored on the system.

The important directories and files belonging to Git after installation are shown in the table below:

Directory or File	Purpose
<code>/usr/bin/git</code>	The main Git executable. This is the actual program that runs when you type git in the terminal.
<code>/usr/lib/git-core/</code>	Internal helper programs that Git uses behind the scenes to perform tasks.
<code>/usr/share/doc/git/</code>	Documentation, changelog and copyright information for the package.
<code>/usr/share/man/man1/</code>	Manual pages for Git, readable by typing man git in the terminal
<code>~/.gitconfig</code>	Per-user configuration file stored in each user's home directory
<code>/etc/gitconfig</code>	System-wide configuration file that applies Git settings to all users on the machine.
<code>.git/</code>	A hidden folder created inside the project when Git <i>init</i> is run. It stores the entire project history.

The most significant ones from the above are `/usr/bin/git`, which is the actual program. The other ones either assist it with helper programs or contain configuration settings or documentation.

User and Group — Why It Matters for Security

If you run a command such as **`ls -la /usr/bin/git`**, you can view the ownership and permissions of the main program file for Git. This is important to understand because it will show you who can use the program, who can edit it, and why this keeps your system secure.

The output of the command looks like this:

`-rwxr-xr-x 1 root root 3045328 Apr 7 2023/usr/bin/git`

Each part of the output tells us something different about the file. The table below explains what each part means:

Part of Output	Meaning
<code>-rwxr-xr-x</code>	The file permission showing who can do what with this file.
<code>1</code>	Number of hard links pointing to this file.
<code>root(first)</code>	The owner of the file – the root user owns it.
<code>root(second)</code>	The group the file belongs to – the root group.
<code>3045328</code>	The size of the file in bytes.
<code>/usr/bin/git</code>	The full path showing where the file is located.

The permission code **`-rwxr-xr-x`** can be further broken down to understand exactly who can do what:

Who	Permission Code	Can Read?	Can Write?	Can execute?
Owner which is root	<code>rwX</code>	Yes	Yes	Yes
Group which is root	<code>r-X</code>	Yes	No	Yes
Everyone else	<code>r-X</code>	Yes	No	Yes

The ownership of the Git program is root, and this means that only the root user has the ability to modify or change the Git program. Users can freely use the Git program, but they are not allowed to modify the program file. This is very important in terms of security, as it ensures that no ordinary user has the ability to change the Git program file, whether by mistake or intentionally. The personal Git configuration for all users is stored in their own personal directory in a file named `.gitconfig`, which is accessible and modifiable only by the user.

Service Management

While other software packages, like Apache, are web servers and MySQL is a database system, Git is not a service program. This means it is not running in the background waiting on network connections or requests to process. Rather, it is a command-line program, which means that you enter a command and it runs until it is complete and then terminates normally. As a result, there is no service to start, stop, enable, or restart.

The table below shows what happens when **common service management commands** are run for **Git**:

Command	Result	What is Means
<code>systemctl status git</code>	Unit not found	Git has no service unit registered
<code>systemctl list-units followed by grep git</code>	No output returned	No Git service is currently running
<code>Sudo systemctl start git</code>	Error message	Git cannot be started as a service

The output of running `systemctl list-units` and piping it to `grep git` is nothing, and this is perfectly correct and expected. This is because the service is not needed because it only runs if the developer is typing a command. This is good design because it does not use memory or CPU time if it is not in use.

The only time that this is not true is if the system is running a Git server, such as `git-daemon`, or if it is running something like `GitLab` or `Gitea`. However, these are not really part of the actual Git program and are more like something that is built upon it.

```
nihal_vats@LAPTOP-07FIALS0:~$ systemctl list-units
UNIT
sys-devices-LNXSYSTM:00-LNXSYBUS:00-ACPI0004:00-MSFT1000:00-c4b741f5\x2d5582\x2d4c98\x2d8f8b\x2d2e082933c396-pci5582:~
sys-devices-LNXSYSTM:00-LNXSYBUS:00-ACPI0004:00-MSFT1000:00-c4b741f5\x2d5582\x2d4c98\x2d8f8b\x2d2e082933c396-pci5582:~
sys-devices-LNXSYSTM:00-LNXSYBUS:00-ACPI0004:00-MSFT1000:00-c4b741f5\x2d5582\x2d4c98\x2d8f8b\x2d2e082933c396-pci5582:~
sys-devices-LNXSYSTM:00-LNXSYBUS:00-ACPI0004:00-MSFT1000:00-ddb05c91\x2da5fa5\x2d4655\x2d963b\x2dec61e1ceb5e2-net-eth0:~
sys-devices-LNXSYSTM:00-LNXSYBUS:00-ACPI0004:00-MSFT1000:00-fd1d2cbd\x2dce7c\x2d535c\x2d966b\x2deb5f811c95f0-host0-ta~
sys-devices-LNXSYSTM:00-LNXSYBUS:00-ACPI0004:00-MSFT1000:00-fd1d2cbd\x2dce7c\x2d535c\x2d966b\x2deb5f811c95f0-host0-ta~
sys-devices-LNXSYSTM:00-LNXSYBUS:00-ACPI0004:00-MSFT1000:00-fd1d2cbd\x2dce7c\x2d535c\x2d966b\x2deb5f811c95f0-host0-ta~
sys-devices-LNXSYSTM:00-LNXSYBUS:00-ACPI0004:00-MSFT1000:00-fd1d2cbd\x2dce7c\x2d535c\x2d966b\x2deb5f811c95f0-host0-ta~
sys-devices-platform-serial8250-tty-ttyS0.device
sys-devices-platform-serial8250-tty-ttyS1.device
sys-devices-platform-serial8250-tty-ttyS2.device
sys-devices-platform-serial8250-tty-ttyS3.device
sys-devices-platform-serial8250-tty-ttyS4.device
sys-devices-platform-serial8250-tty-ttyS5.device
sys-devices-platform-serial8250-tty-ttyS6.device
sys-devices-platform-serial8250-tty-ttyS7.device
sys-devices-virtual-block-ram0.device
sys-devices-virtual-block-ram1.device
sys-devices-virtual-block-ram10.device
sys-devices-virtual-block-ram11.device
sys-devices-virtual-block-ram12.device
sys-devices-virtual-block-ram13.device
sys-devices-virtual-block-ram14.device
sys-devices-virtual-block-ram15.device
sys-devices-virtual-block-ram2.device
```

Update Model — How Security Patches Reach Your System

Since Git is managed by Ubuntu's package manager, it receives updates as part of the regular update cycle. When the open-source community for Git finds a bug or a security flaw, they release a new version of the program. The Ubuntu community then takes this updated version and makes it available for all users by making it available through the usual update commands.

The full process from when a patch is created to when it reaches the user's machine is shown below:

Step	What Happens
Step 1	Git developers find and fix a bug or security vulnerability in the code
Step 2	A new version is released on git-scm.com along with the full source code
Step 3	Ubuntu maintainers review and package the new version for the apt repository
Step 4	The update becomes available in Ubuntu's official package repository
Step 5	The user runs <code>sudo apt update</code> to refresh the list of available packages
Step 6	The user runs <code>sudo apt upgrade git</code> to download and install the update

One of the strongest aspects of using open-source software is that security updates are completely transparent. This means that when an update is released for the Git software, anyone in the world can look at it and see exactly what was changed and why it was changed. There are no hidden changes or updates. The entire history of every single change ever made to the Git software is out in the open in its source code. This is not something that proprietary software can do, and it is one of the biggest reasons why open-source software is used in professional environments.

5. Part C — The FOSS Ecosystem

No open-source project is an island. All software relies upon other tools and communities that came before it. And that is true of Git. This section looks at the larger ecosystem that surrounds it: what it depends upon, what it has spawned, where it sits in the world of open source, and who is in charge of it all

Dependencies: What Other Open-Source Tools Git Depends On?

Git works in conjunction with other open-source software packages. Although Linus Torvalds designed Git to be simple, it depends on a number of other open-source software packages to work properly. These software packages are called dependencies, and they are essentially other open-source software packages that Git depends on to work properly.

The table below shows the main tools and libraries that Git depends on:

Dependency	Type	What it does for Git
C Standard Library (glibc)	Core library	Git is written in C and needs glibc to handle basic operations like memory, input and output
OpenSSL or GnuTLS	Security library	Handles encrypted HTTPS connections when pushing or pulling from remote repositories
libcurl	Network library	Manages all HTTP and HTTPS communication between Git and remote servers like GitHub
zlib	Compression library	Git compresses all stored objects to save space. zlib handles this compression and decompression
POSIX Shell (bash)	Shell environment	Many of Git's built in helper scripts are written in shell and require bash to run
Perl	Scripting language	Some older Git tools and helper scripts are written in Perl
SSH (OpenSSH)	Secure connection tool	Used when connecting to remote repositories over SSH instead of HTTPS
gettext	Internationalisation library	Allows Git's messages and output to be translated into different languages

All of these dependencies are also open source. This is a very important point: Git is not just open source, it is built entirely on top of other open-source tools. This is what it means to **"stand on the shoulders of giants."** Every layer of this system, from the C library at the bottom to the shell scripts at the top, is open and free.

What Git Has Enabled And Inspired: The Ecosystem It Created

Git did not just solve a problem for the version control community. Git actually changed the way software development works as a whole. The tools and platforms that Git has inspired and enabled are, arguably, among the most important software development infrastructure ever.

The major platforms, tools and projects that Git has directly created or inspired are:

GitHub

Code Hosting Platform.

Built entirely on top of Git. Launched in 2008, bought by Microsoft in 2018 for 7.5 billion dollars. Hosts over 100 million repositories.

GitLab

Code Hosting and DevOps Platform.

A fully open-source alternative to GitHub. Used by thousands of companies for private self hosted repositories.

BitBucket

Code Hosting Platform

Built on Git, owned by Atlassian. Popular in enterprise environments alongside Jira.

Gitea

Lightweight Self Hosted Git.

A small open source Git server anyone can run on their own machine or server.

Linux Kernel Development

Open Source Project

The Linux kernel itself moved to Git in 2005 and still uses it today for all development.

The biggest thing that Git made possible is the notion of open-source collaboration. Traditionally, if someone wanted to contribute to an open-source project, it had always been hard. They would need permission, access, etc. With the advent of Git, the notion of “forking” was born. This is where you can create your own copy of the project, make changes, and then send a pull request to the original project with your changes.

This notion has completely revolutionized the world of open-source. Now, millions of developers can contribute to projects they have not officially been invited to, due to the distributed model that Git has made possible.

Git in the LAMP Stack / Web Context

The combination of Linux, Apache, MySQL, and PHP is commonly referred to by the mnemonic LAMP. This is one of the most popular collections of free software used to power websites and web applications. Git is not technically part of the LAMP stack, though it is an integral part of the application development process using the LAMP stack.

The table below shows the LAMP stack components and where Git fits alongside them:

Component	Role in the Stack	Relationship With Git
Linux	Operating System that runs everything.	Git is installed on Linux and manages all code running on it
Apache	Web server that handles HTTP requests	Git is used to deploy and update Apache configuration files and web application code
MySQL	Database that stores application data	Database migration scripts and schema files are tracked and versioned using Git
PHP/Python	Programming language that runs the application	All application source code is stored, tracked, and collaborated on through Git
Git	Version control	Sits across the entire stack — tracks every change to code, config files, and scripts

In a real-world development process, a developer would write code on his or her machine, commit the code to Git, push it to a remote repository like GitHub or GitLab, and then the automated pipeline would deploy it to the Linux server with Apache, MySQL, and PHP. Git is the thread that connects the developer's machine with the server. Without Git, it would be very difficult to manage code changes with a team and deploy applications reliably.

The Git Community: Who Governs It?

It is not owned by any company. It is managed by a global network of volunteers and professional developers. They all work together to make sure it is reliable, safe, and updated. This is what helps us understand the success and safety of Git over two decades.

Original Creator — Git was created by Linus Torvalds in April 2005. But very soon after that, the actual maintenance of the source code of the Git version control system was handed over to others by Linus, as his primary focus was the kernel.

Current Maintainer — The primary person maintaining the source code of the version control system is Junio C Hamano, and this role is still in his hands. His task is to review all the patches that are submitted by the contributing members and decide what patches are to be merged into the actual source code of the system. His contributions have been the backbone of the stability of the system for the last two decades.

How Contributions are Made to the System — Contributions to the system are made by anyone in the world. A person develops a patch, which is a small piece of code that is supposed to fix a bug or add a feature to the system, and sends it to the mailing list. The patches are reviewed by the primary person in charge, i.e., Junio, before they are accepted into the actual source code of the system. This way, the version control system is able to utilize the ideas of thousands of people all over the world.

Source Repository — The source code of the version control system is hosted at git.kernel.org. The source code is also hosted at GitHub and is available at github.com/git/git. A person can read every line of the source code of the system, every change that has been made to the source code,

and understand the actual working of the system.

Mailing List — The actual place where the development of the system is taking place is the mailing list at git@vger.kernel.org.

The way in which Git is managed is what is called a benevolent maintainer model. This is where one person, currently Junio C Hamano, has the final say in what is included in the Git system. However, the entire process is open, and everything is done in public via the mailing list. This is very different from a commercial product, where decisions are not made in public. However, thanks to the open nature of the project, Git has been able to produce high-quality releases for twenty years without any major issues or splits in the project.

Conferences and Learning Resources:

The Git and open-source community can be found through their meetings, online forums, and tutorials. These are venues where the community shares their plans for the future of Git, where knowledge is being shared among experienced and novice programmers, and where everyone is invited to join.

Git Merge - This is the official annual conference of the Git community. Here, all the contributors, maintainers, and users get together and share their ideas and plans for the future of the development of the Git system. This is the most important event in the calendar of the Git community.

FOSDEM - FOSDEM stands for Free and Open-Source Developers European Meeting. It is the biggest free open-source event in the world and is held every year in February in Brussels, Belgium. The members of the Git community are regular speakers and presenters at FOSDEM, and it is the biggest event for the whole open-source community.

GitHub Universe - This is the official annual conference of GitHub, and it is all about the workflows and open-source community related to the Git system. GitHub is built entirely around the Git system, and it is one of the most important events in the calendar of the Git community.

Stack Overflow – This is the most popular online community among programmers. This community contains millions of questions and answers related to Git. This is where every programmer looks first whenever they encounter a problem related to Git.

What sets the Git community apart is that it's completely open. All of its decisions, all of its patches, all of its discussions are out in the open for anyone who's interested to read. This is not a philosophical choice, by the way. This is what makes the community trustworthy. Developers and companies all over the world rely on Git for their most critical work, and they can do this because they know exactly who's behind it, exactly how it's made, and exactly where it's headed. This is open source as it's meant to be.

6. Part D — Open-Source vs Proprietary

No tool exists in a vacuum, and Git is no exception. In order to effectively discuss Git as a viable option for professionals, it is necessary to compare it to the best proprietary alternatives. In this section, Git is compared to Microsoft's Team Foundation Version Control, a part of Azure DevOps, and Perforce's Helix Core, a very popular solution for big businesses and game development studios. The comparison is honest and balanced since open-source is not always better.

The table below compares Git directly against its proprietary alternatives across the most important dimension for a real-world deployment:

Dimension	Git	Microsoft TFVC	Perforce Helix Core
Source Code Access	Fully Open	Closed Source	Closed Source
Security Auditing	Anyone can inspect the code	Only Microsoft can audit	Only Perforce can audit
Freedom to Modify	Full freedom under GPL v2	No freedom	No freedom
Cost	Completely free	Free up to 5 users, paid beyond that	Very expensive, thousands of dollars per year
Distribution Model	Distributed, works offline	Centralized – needs server	Centralized – needs server
Performance	Very fast for code	Slower with large repos	Fast with large binary files
Support	Community driven	Paid Microsoft support	Paid perforce support
License	GPL v2 – permanently open	Proprietary	Proprietary

When Proprietary Might Be The Better Choice?

A balanced comparison involves recognizing that Git isn't necessarily the best choice for every problem. There are particular situations where proprietary systems have advantages over Git.

Large binary files – Git does a poor job with very large binary files, for example, 3D models, game assets, and videos. Game development studios have found this to be a problem where Perforce excels compared to Git.

Enterprise-level support contracts – Some organizations require a legal support contract with guaranteed response times. Microsoft and Perforce have this, but the Git community

does not.

Microsoft integration – Organizations heavily invested in Microsoft products, for example, Visual Studio and Azure, find TFVC integrates more smoothly out of the box.

Overall Verdict

After comparing it against its proprietary counterparts, it's safe to say that for the vast majority of software development projects, Git is the better choice. Not only is it free, fast, and distributed, but it also boasts the largest community of developers anywhere.

The complete lack of vendor lock-in means that no company can ever hold your code hostage. The only situations where a proprietary alternative is warranted are very specific, such as a game development studio that works with massive binary codebases, a company that requires very specific support from their source control solution, or a team that's already invested heavily into the Microsoft ecosystem. In every other case, Git is not only good, it's the very best solution available.

Would I Contribute to Git? Yes. Any contribution, even as simple as correcting a piece of documentation, reporting a bug, or answering a question on Stack Overflow, is a contribution. Open source succeeds because people give back, and Git has given the world something incredible.

7. Shell Script Tasks

Five shell scripts accompany this report. Each demonstrates practical Linux skills connected to the open-source software examined above.

Script 1 — System Identity Report

In this script, we make use of shell variables to store student and system information. The commands used in this script include command substitution using dollar signs and brackets to execute system commands like `uname`, `lsb_release`, `whoami`, and `date` and directly insert their results into variables using dollar signs and brackets. The `echo` command is used to display all this in a formatted layout as required in a welcome screen. This is an example of using shell variables, command substitution, and formatted output, which are the basic ideas behind Unit 1 and Unit 2 shell scripting.

```
THIS message is shown once a day. To disable it,
/home/nihal_vats/.hushlogin file.
nihal_vats@LAPTOP-07FIALS0:~$ nano script1.sh
nihal_vats@LAPTOP-07FIALS0:~$ bash script1.sh
Student : Nihal Vats
OS      : Ubuntu
Kernel  : 6.6.87.2-microsoft-standard-WSL2
User    : nihal_vats
Date    : Sun Mar 29 16:54:53 UTC 2026
License : GPL v2 (Linux Kernel)
nihal_vats@LAPTOP-07FIALS0:~$ |
```

Script 2 — FOSS Package Inspector

In script 2, an "if-then-else" statement is used to check whether the Git package is installed in the system by using the `dpkg` command. If the Git package is installed, `grep` is used in conjunction with a pipe to filter and display the relevant details from the entire output of the `dpkg` command. A `case` statement is used to match the name of the package and display a one-line philosophy note about it. This script shows conditional logic in "if-then-else" statements, pattern matching in `case` statements, and output filtering using pipes and `grep`.

```
nihaI_vats@LAPTOP-07FIALS0:~$ nano script2.sh
nihaI_vats@LAPTOP-07FIALS0:~$ bash script2.sh
git is installed.
git version 2.43.0
Git: built in 10 days, changed software forever. Licence: GPL v2
nihaI_vats@LAPTOP-07FIALS0:~$
```

Script 3 — Disk and Permission Auditor

In Script 3, a for loop is used to iterate through a list of important directories of the system and report their permission and disk usage. Within the for loop, an if statement is used to check whether a directory exists before it can be read. The ls command, along with awk, is used to get information about the permission and ownership of the directories, and the du command, along with cut, is used for disk usage. At the end of the script, it specifically searches for the configuration file of Git and displays its permission.

```
niha1_vats@LAPTOP-07FIALS0:~$ nano script3.sh
niha1_vats@LAPTOP-07FIALS0:~$ bash script3.sh
/etc => drwxr-xr-x root | 4.3M
/var/log => drwxrwxr-x root | 51M
/home => drwxr-xr-x root | 52K
/usr/bin => drwxr-xr-x root | 95M
/tmp => drwxrwxrwt root | 28K
niha1_vats@LAPTOP-07FIALS0:~$ |
```

Script 4 — Log File Analyzer

In script 4, a log file path and a keyword are passed as command-line arguments using \$1 and \$2. The script checks if the file exists and is empty using an if statement with a file test operator. The while read loop is used to read the log file line by line. Inside this loop, another if statement checks each line in the log file for a specified keyword using grep. The variable acts as a counter to keep track of how many lines contain the keyword. At the end, it prints out how many lines contained the keyword and also prints out the last 5 lines using grep and tail commands together.

```
niha1_vats@LAPTOP-07FIALS0:~$ nano script4.sh
niha1_vats@LAPTOP-07FIALS0:~$ bash script4.sh /var/log/dpkg.log error
Keyword 'error' found 39 time(s) in /var/log/dpkg.log
2026-02-10 00:56:02 status unpacked liberror-perl:all 0.17029-2
2026-02-10 00:56:32 configure liberror-perl:all 0.17029-2 <none>
2026-02-10 00:56:32 status unpacked liberror-perl:all 0.17029-2
2026-02-10 00:56:32 status half-configured liberror-perl:all 0.17029-2
2026-02-10 00:56:32 status installed liberror-perl:all 0.17029-2
```

Script 5 — Open Source Manifesto Generator

The script in Script 5 employs the `read` command with the `-p` flag to prompt the user to enter three answers interactively and store them in variables. The script then employs string concatenation using `echo` and the append operator to construct a manifesto paragraph tailored to the user and write it to a text file using redirect operators. The `whoami` command is embedded in the filename to make the resulting file unique to the user. The `date` command is used to include a date in the manifesto. The `cat` command is used to display the saved file back to the user. The script also illustrates the alias concept in shell scripting in a comment where it suggests how it could be shortened to a one-word command. This script illustrates user interaction using `read`, file writing using redirect operators, string concatenation, and the alias concept.

```
nihal_vats@LAPTOP-07FIALS0:~$ nano script5.sh
niha_vats@LAPTOP-07FIALS0:~$ bash script5.sh
A tool you use daily: Git
One word for freedom: Free
Something you'd build freely: MindMate
Open Source Manifesto
I use Git every day – free software anyone can read and change.
Freedom to me means Free.
One day I'll build MindMate and share it the same way.
Signed: niha_vats | 29 March 2026
niha_vats@LAPTOP-07FIALS0:~$ |
```


8. References

The following sources were used for research and understanding of topics covered in this report. All written contents is in my own words.

1. [GNU Project](https://gnu.org/philosophy/free-sw.html) – The Free Software Definition. gnu.org/philosophy/free-sw.html
2. [Open Source Initiative](https://opensource.org/osd) – The Open-Source Definition. opensource.org/osd
3. [Eric S. Raymond](https://catb.org/~esr/writings/cathedral-bazaar) – The Cathedral and the Bazaar. catb.org/~esr/writings/cathedral-bazaar
4. [Choose a License](https://choosealicense.com) – License comparisons. choosealicense.com
5. [SPDX License List](https://spdx.org/licenses/GPL-2.0-only.html) – Full GPL v2 license text. spdx.org/licenses/GPL-2.0-only.html
6. [Git Official Website](https://git-scm.com) and [Pro Git Book](https://git-scm.com/book). git-scm.com and git-scm.com/book
7. [Git Source Repository and commit history](https://github.com/git/git). github.com/git/git
8. [Shotts W.](https://linuxcommand.org) – The Linux Command Line. linuxcommand.org
9. [GNU Bash Manual](https://gnu.org/software/bash/manual) – Bash scripting reference. gnu.org/software/bash/manual
10. [Ubuntu Official Documentation](https://ubuntu.com/server/docs) – apt package management. ubuntu.com/server/docs
11. [Linux Manual Pages](https://man.cx) – Command reference. man.cx
12. [Software Freedom Conservancy](https://sfconservancy.org) – Git governance. sfconservancy.org
13. [Microsoft Azure DevOps](https://docs.microsoft.com/azure/devops) – TFVC documentation. docs.microsoft.com/azure/devops
14. [Perforce Helix Core](https://perforce.com/products/helix-core) – Enterprise version control. perforce.com/products/helix-core
15. [Stack Overflow Developer Survey 2023](https://survey.stackoverflow.co/2023) – Git usage statistics. survey.stackoverflow.co/2023